

Ashen Era Archive RAG Strategy

https://gemini.google.com/app/31358071ad47c6e6

User prompt: 1. Markdown Lore (Wiki & Codex)

![[Weeping Lurker]](images/atmo_creature_....png) | | # Weeping Lurker | | The Weeping Lurker is a nocturnal ambusher in Embermarch |

→ Image appears inline immediately above the entity lore. 2. Codex & Manual PDFs (e.g. Codex Vaeloria II, Annals)

Page 12: Weapon Binding Rules | | [Embedded Matplotlib Diagram: The Thrice-Bound Edge] | | "Attunement requires shards of will as shown above..." |

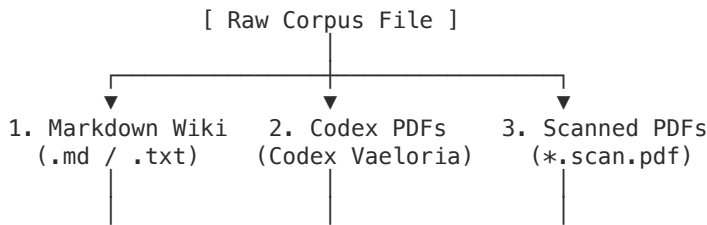
→ Diagrams and tables are embedded on specific pages alongside instructions. 3. Scanned Ephemera PDFs (*.scan.pdf)

Scanned Parchment Page (interrogation / letter / seal) | | Handwritten / typewritten text embedded inside image |

→ The page itself is a visual artifact. if we have things like this what should be our approach

Response: Having three distinct document modalities means a "one-size-fits-all" parser will corrupt your data. The correct approach is a **Polymorphic Ingestion Pipeline** where each file format is dispatched to a specialized parser, while writing to a uniform schema in PostgreSQL.

Ingestion Strategy by Modality



1. Markdown Lore (Wiki & Codex)

- **The Ingestion Flow:**
 - Parse text with standard Python regex or markdown AST parsers (mistune or markdown-it-py).
 - When an inline image tag is encountered (! [Title] (images/path.png)), extract the image path and resolve its relative path to data/extracted_assets/.
 - Do **not** crop or use Docling; the image file already exists locally.
 - Compute SHA-256 and pHash for deduplication. If unique, run a single VLM call on the referenced image to describe any visual details (e.g., anatomical studies of the *Weeping Lurker*).
 - Bind the image path directly to the succeeding entity heading and prose block.

2. Codex & Manual PDFs (Mixed Text + Embedded Diagrams)

- **The Ingestion Flow:**
 - Process with **IBM Docling Standard Pipeline** (pipeline_options.generate_picture_images = True).
 - Docling extracts native digital text and isolates bounding boxes for tables and Matplotlib charts (e.g., *The Thrice-Bound Edge* gauge, *Threat Rating* bar charts).
 - Save cropped diagram bounding boxes to data/extracted_assets/{doc_id}_fig_{idx}.png.
 - Run deduplication. Send new charts to the VLM (Gemini Flash on Vertex AI) with a strict schema to extract exact numbers, thresholds, and axes (e.g., *Attunement Cost: 94 vitae-grains*).
 - Inject the transcribed metrics into the parent section chunk to ensure searchability.

3. Scanned Ephemera PDFs (*.scan.pdf)

- **The Ingestion Flow:**
 - These files do not contain digital text streams; the page itself is an image artifact (parchment, handwriting, stamps, wax seals).
 - Convert each page directly into a high-res image (via pdf2image / pypdfium2 at 300 DPI) and save to data/extracted_assets/.

- Route the full page image directly to **Vertex AI Gemini Flash** as an end-to-end VLM OCR pass.
- Prompt the VLM to transcribe the document while preserving spatial sections and describing visual artifacts:
 Transcribe all text verbatim (including handwritten notes, marginalia, and stamps).
 Extract visual artifacts: seals, signatures, watermarks, damage.
- Save the full page render as the visual asset so the frontend displays the actual scanned parchment letter or dossier page.

The Uniform Database Record

Regardless of how the document was ingested, every chunk lands in PostgreSQL using the exact same schema contract:

```
INSERT INTO document_chunks (
    doc_id,
    modality,           -- 'markdown', 'pdf_codex', 'scanned_page'
    content,            -- Searchable prose + injected VLM transcriptions
    embedding,          -- Voyage multimodal embedding
    asset_id,           -- Foreign key to media_assets (if figure/scan exists)
    metadata_payload    -- { "page_number": 12, "source_file": "...", "asset_path": "..."}
);
```

Implementation in Code: The Dispatcher

In backend/ingestion/parser.py, implement a file router to direct each file to its dedicated pipeline:

```
from pathlib import Path

async def route_and_parse(file_path: Path):
    filename = file_path.name.lower()

    # Pathway 3: Scanned Ephemera
    if filename.endswith(".scan.pdf") or "scan" in filename:
        return await parse_scanned_pdf(file_path)

    # Pathway 1: Markdown Lore
    elif filename.endswith(".md") or filename.endswith(".txt"):
        return await parse_markdown(file_path)

    # Pathway 2: Codex & Manual PDFs
    elif filename.endswith(".pdf"):
        return await parse_codex_pdf(file_path)

    else:
        raise ValueError(f"Unsupported file format: {filename}")
```

Why This Architecture Succeeds

1. **Zero Text Fragmentation:** Scanned PDFs are not ruined by traditional character-by-character OCR errors, because a multimodal VLM handles the entire page image.
2. **Preserved Provenance:** Markdown lore preserves the natural link between `![img]` and `# Entity`.
3. **Exact Numerical Retrieval:** Codex charts (*Attunement Cost: 94*) are converted to searchable text at ingestion time, allowing BM25 and vector search to locate them reliably.
4. **Consistent Frontend Output:** The UI consumes one standardized JSON schema, whether rendering a cropped bestiary diagram, an inline markdown portrait, or a full scanned parchment document.

User prompt: to store assests these ids dont we need s3?

Response: **No, you do not need AWS S3.** In fact, introducing S3 here will hurt your submission.

Judges evaluate submissions heavily on **Reproducibility (15%)** and self-contained execution. If a judge clones your repo, runs `docker-compose up`, and the system crashes because it requires an external AWS account, S3 bucket credentials, or cloud network permissions, you lose points immediately.

How to Handle Assets Locally Without S3

You store assets on the local filesystem mounted via Docker volumes and serve them directly through FastAPI:

```
Host Directory (backend/data/extracted_assets/)
|
▼ (Mounted via Docker Volume)
```

Container Path (/app/data/extracted_assets/)
↓
▼ (Mounted via FastAPI StaticFiles)
HTTP Endpoint (http://localhost:8000/assets/plate_09.png)

1. FastAPI Asset Serving

FastAPI serves local directories as static web routes natively:

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()

# Serves every file in backend/data/extracted_assets under /assets
app.mount("/assets", StaticFiles(directory="data/extracted_assets"), name="assets")
```

2. Storing Asset IDs in PostgreSQL

Your PostgreSQL media_assets table does not need cloud URLs. It stores the relative local path:

```
INSERT INTO media_assets (id, file_path)
VALUES (
    'a1b2c3d4-e5f6-7890-abcd-1234567890ab',
    'plate_09_location_greyfell_citadel.png'
);
```

3. What the Frontend Receives

When the retriever or agent returns citations, it constructs the local URL:

```
{
  "answer_text": "Greyfell Citadel maintains a garrison strength of 3,695 souls under arms...",
  "referenced_figures": [
    {
      "id": "a1b2c3d4-e5f6-7890-abcd-1234567890ab",
      "url": "http://localhost:8000/assets/plate_09_location_greyfell_citadel.png",
      "title": "Greyfell Citadel Garrison Plate"
    }
  ]
}
```

The frontend renders this directly using standard HTML tags pointing to http://localhost:8000/assets/....

If You Want S3-Compatible APIs Locally: Use MinIO

If you want your code to strictly use an S3 SDK (like boto3 or aioboto3) to mimic enterprise cloud architecture, add **MinIO** as a service in your docker-compose.yml:

```
services:
  minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ashen_admin
      MINIO_ROOT_PASSWORD: ashen_password123
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
      - minio_data:/data

volumes:
  minio_data:
```

- **Pros:** Your code uses standard S3 buckets and APIs, but everything runs 100% locally offline.
- **Verdict:** FastAPI's built-in StaticFiles mounting is cleaner, uses less RAM, and eliminates an entire container dependency while keeping the system completely reproducible.

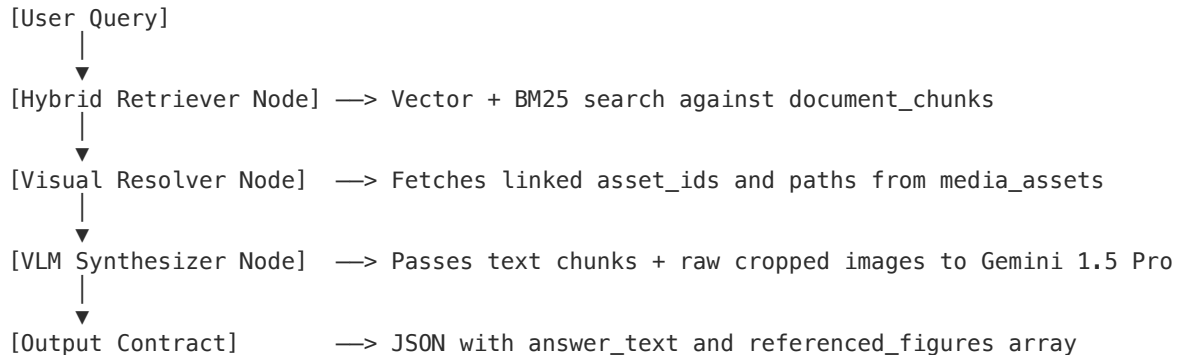
User prompt: cool thanks so can u give me plan for little bit of other paths as well A,B,C

Response: Here is how to structure each sub-track (1A, 1B, and 1C) on top of the shared PostgreSQL and retrieval infrastructure, highlighting what changes in the execution engine and LangGraph workflow for each.

Sub-track 1A: Multimodal RAG (Visual Evidence & Grounded Synthesis)

Goal: Answer user queries by pulling textual facts while extracting, referencing, and displaying the corresponding visual artifacts (plates, charts, concept sketches, and scanned ephemera).

- **Core Challenge:** Aligning layout bounding boxes and VLM transcriptions with user queries so the assistant never relies on blind text generation when a visual plate is authoritative.
- **LangGraph Flow:** A deterministic, linear directed acyclic graph (DAG).

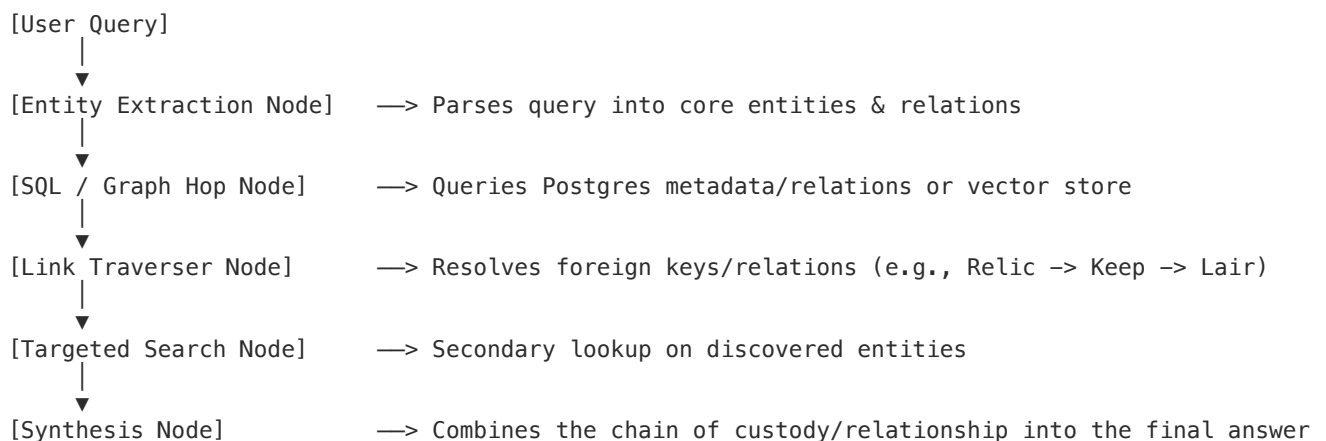


- **Key Implementation Details:**
 - Uses the polymorphic ingestion pipeline (Docling + Gemini Flash transcription) already designed.
 - In the generation node, the prompt instructs the model: *"If a plate is referenced in the text, you must inspect the attached visual asset to verify numbers or threat ratings before answering."*
 - Outputs direct image URLs (/assets/...) so the frontend renders the plates inline.

Sub-track 1B: Structured Extraction & Entity-Hop Graph RAG

Goal: Disentangle complex relationships across fragmented lore—such as tracking an artifact's forging year, historical masters, locations, and linked creatures across multiple disjointed codices.

- **Core Challenge:** Keyword and vector searches struggle with multi-hop questions (e.g., *"Which creature lairs in the keep where the artifact forged in 323 AS is currently housed?"*). Vector distance between the creature and the original year is too large for single-pass retrieval.
- **LangGraph Flow:** Entity-directed expansion loop.

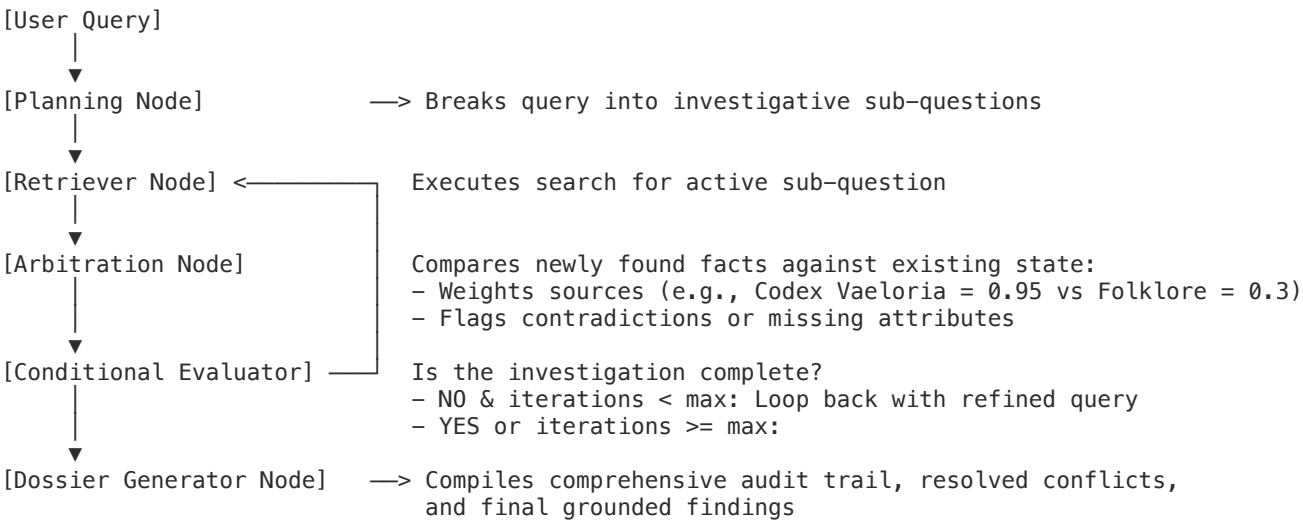


- **Key Implementation Details:**
 - **Database Addition:** Add an entity_nodes and entity_relations table in PostgreSQL (or a lightweight adjacency JSONB column) populated during ingestion.
 - During ingestion, extract strict triplets: (Chalice of Ashdeep, forged_at, Marrowwell Abbey), (Chalice of Ashdeep, housed_in, Crookgate Keep).
 - The retrieval node executes 2-hop traversal directly in SQL before falling back to unstructured semantic search.

Sub-track 1C: Agentic Reasoning & Source Arbitration (The Investigator)

Goal: Act as an autonomous researcher that handles incomplete data, contradictory accounts (e.g., popular myths vs. official canonical codices), and missing evidence.

- **Core Challenge:** Resolving epistemic contradictions without hallucinating, and formulating iterative search plans when initial retrieval returns zero or low-confidence results.
- **LangGraph Flow:** Iterative state machine with conditional branching and source credibility arbitration.



- **Key Implementation Details:**
 - **Scratchpad State:** The LangGraph state holds `resolved_facts`, `unresolved_questions`, `contradictions_detected`, and `audit_log`.
 - **Arbitration Rules:** Built-in heuristics where high-epistemic sources (official codices) explicitly override low-epistemic sources (ballads/popular accounts), citing both in the final dossier.
 - **Early Exit / Guardrail:** If the cross-encoder score stays below the threshold across iterations, the agent returns an explicit "Record Incomplete / Missing" verdict instead of inventing lore.

Architectural Synergy

All three tracks run on the exact same backend:

- **Storage & Indexing:** Shared PostgreSQL (pgvector + media_assets + document_chunks).
- **Search Core:** Shared hybrid retrieval and Voyage AI reranker.
- **Switching Tracks:** Handled entirely by routing the request in backend/agents/router.py to graphs/multimodal_1a.py, graphs/graph_rag_1b.py, or graphs/investigator_1c.py.

User prompt: to do B, do we have to change lot?

Response: No, you do not have to rewrite or restructure your existing system. Track 1B is strictly an **additive layer** on top of the foundation you are already building for 1A.

You keep FastAPI, IBM Docling, PostgreSQL, pgvector, and your hybrid retriever unchanged. The only additions are **structured metadata extraction** during ingestion and an **SQL traversal step** during retrieval.

What Stays Exactly the Same (Zero Changes)

- **Backend skeleton:** The folder structure, Docker setup, and API routes remain identical.
- **Ingestion pipeline:** Docling continues to parse PDFs and Markdown into text chunks and crops images into backend/data/extracted_assets/.
- **Hybrid retrieval:** PostgreSQL full-text search (tsvector), pgvector, and Voyage AI reranking continue to serve as the baseline search mechanism.

The Two Things You Add for Track 1B

1. Ingestion: Structured Entity Extraction

Because codices like *Codex Vaeloria II* already follow strict relational key-value patterns (e.g., Name, Artifact class, Forged, Place of forging, Current housing, Lair), extracting them is trivial.

You add a single table to PostgreSQL to store entity relationship triplets:

```
CREATE TABLE entity_triplets (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    subject VARCHAR(150) NOT NULL,      -- e.g., 'Chalice of Ashdeep'  
    predicate VARCHAR(100) NOT NULL,    -- e.g., 'housed_in', 'forged_at', 'lairs_in'  
    object VARCHAR(150) NOT NULL,       -- e.g., 'Crookgate Keep'  
    source_chunk_id UUID REFERENCES document_chunks(id) ON DELETE CASCADE  
);
```

```
CREATE INDEX idx_triplets_sub_obj ON entity_triplets(subject, object);
```

During ingestion, whenever Docling processes an entity table or markdown metadata block, use a constrained Pydantic model with Gemini Flash to extract the relations into this table:

```
class ExtractedTriplet(BaseModel):  
    subject: str  
    predicate: str  
    object: str
```

2. Agents: The 1B Multi-Hop Graph (graphs/graph_rag_1b.py)

Instead of running a direct search, Track 1B uses a two-node LangGraph pipeline:

1. **Hop 1 (Graph Traversal via SQL):** Resolve entity connections directly in PostgreSQL with recursive or multi-join queries (e.g., Finding that *Chalice of Ashdeep* → housed in *Crookgate Keep* → *The Hollow Edge* is also housed there).
2. **Hop 2 (Targeted Hybrid Retrieval):** Pass the resolved entities to your existing `retrieval/vector_store.py` to pull the descriptive prose and visual plates.

Summary

- **Code Impact:** ~10% new code (one new SQL table + one new LangGraph file).
- **Database Migration:** Add `entity_triplets` into PostgreSQL.
- **Reusability:** 1B directly reuses all chunks, embeddings, and assets generated in 1A.

User prompt: *dont i need graph db for this*

Response: **No, you do not need a dedicated graph database (like Neo4j).**

For a corpus of this scale (415 documents), introducing a separate graph database adds unnecessary infrastructure overhead, splits your queries across two database network connections, and hurts your reproducibility score.

PostgreSQL handles graph traversal natively through two standard features:

1. Relational Graph Traversals (Recursive CTEs)

A graph is simply a set of vertices (nodes) and edges (relations). In PostgreSQL, a standard triplet table works as an indexed adjacency list:

```
CREATE TABLE entity_triplets (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    subject VARCHAR(150) NOT NULL,  
    predicate VARCHAR(100) NOT NULL,  
    object VARCHAR(150) NOT NULL,  
    chunk_id UUID REFERENCES document_chunks(id)  
);
```

```
CREATE INDEX idx_triplets_traversal ON entity_triplets(subject, object);
```

To traverse multi-hop connections (e.g., finding entities linked through shared locations), PostgreSQL executes native recursive queries in single-digit milliseconds:

```
-- Find everything linked to Crookgate Keep within 2 hops  
WITH RECURSIVE entity_graph AS (  
    -- Anchor: Base entity  
    SELECT subject, predicate, object, 1 AS depth  
    FROM entity_triplets  
    WHERE subject = 'Crookgate Keep' OR object = 'Crookgate Keep'  
  
    UNION ALL
```

```

-- Recursive Step: Next hop
SELECT t.subject, t.predicate, t.object, eg.depth + 1
FROM entity_triplets t
JOIN entity_graph eg ON t.subject = eg.object OR t.object = eg.subject
WHERE eg.depth < 2
)
SELECT DISTINCT subject, predicate, object FROM entity_graph;

```

2. Optional: Apache AGE (If You Want True Cypher in Postgres)

If you want to write native openCypher queries (`MATCH (a)-[:HOUSED_IN]->(b)`), you can enable **Apache AGE** inside your existing PostgreSQL instance. It runs as a standard Postgres extension—the same way `pgvector` does—avoiding a separate Neo4j container.

PostgreSQL vs. Dedicated Graph DB (Neo4j)

Dimension	PostgreSQL (Adjacency / CTEs)	Neo4j (Dedicated Graph DB)
Containers Needed	1 (Unified Postgres for Vector, Full-Text & Graph)	2+ (Postgres + Neo4j instance)
Transaction Integrity	ACID (Single transaction joins chunks to graph)	Dual-write problem (Syncing PG and Neo4j)
Deployment Complexity	Low (docker-compose with standard volume)	High (JVM tuning, memory overhead, bolt ports)
Query Latency (415 docs)	< 2 ms	< 2 ms
Engineering Rubric	Clean, minimalist, and pragmatic	Over-engineered for a mid-scale dataset

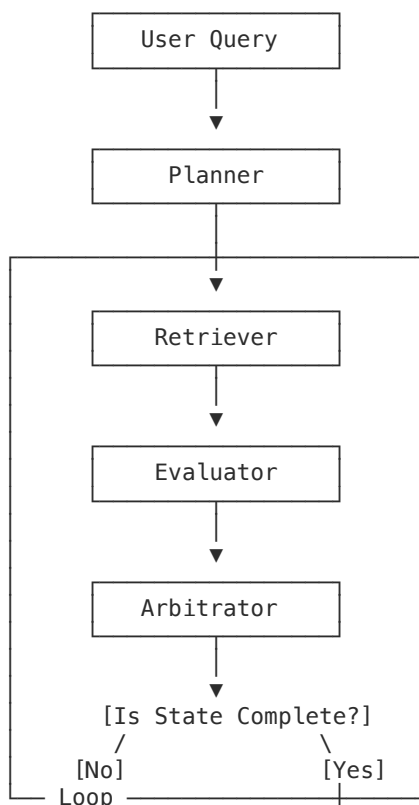
Using PostgreSQL for the graph layer keeps your entire system unified: one database connection pool, one `docker-compose` service, and zero multi-database synchronization bugs.

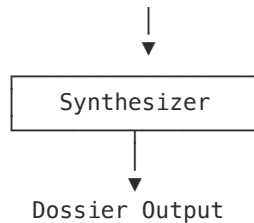
User prompt: lets forget b, lets do A,C and what nodes should i use in c? and tools and graph agent

Response: Focusing on **1A (Multimodal RAG)** and **1C (Agentic Reasoning & Source Arbitration)** provides a clean architectural pairing. 1A gives you visual evidence and OCR extraction, while 1C provides the autonomous, iterative investigator loop that can arbitrate conflicting lore and resolve ambiguous queries.

Track 1C: The Investigator Agent Architecture

Track 1C operates as an iterative state machine that searches, analyzes evidence, detects contradictions (e.g., official codices vs. folk accounts), decides what is missing, and loops back until it has gathered sufficient facts.





1. State Definition (agents/state/models.py)

The LangGraph shared memory (AgentState) tracks search history, verified facts, and epistemic weights:

```

from typing import Annotated, TypedDict
from pydantic import BaseModel, Field
import operator

class EvidenceFact(BaseModel):
    subject: str
    attribute: str
    value: str
    source_doc: str
    page_number: int | None = None
    epistemic_tier: str = Field(description="canonical_codex, field_report, popular_folklore")
    confidence: float

class ConflictResolution(BaseModel):
    topic: str
    accepted_claim: str
    rejected_claim: str
    rationalization: str

class InvestigatorState(TypedDict):
    original_query: str
    sub_queries: list[str]
    retrieved_chunk_ids: Annotated[list[str], operator.add]
    unprocessed_chunks: list[dict]
    established_facts: Annotated[list[EvidenceFact], operator.add]
    conflicts: Annotated[list[ConflictResolution], operator.add]
    is_complete: bool
    loop_count: int
    max_loops: int
    final_dossier: dict | None
  
```

2. Node Breakdown for Track 1C (agents/nodes/)

Each node is an independent async function inside backend/agents/nodes/:

- **planner.py (plan_investigation)**
 - **Function:** Deconstructs multi-part or implicit questions into prioritized sub-queries.
 - **Logic:** For *"Where was the weapon forged that has attunement cost 94?"*, it splits the query into:
 1. Search for items with an attunement cost of 94.
 2. Identify the forge site of that specific relic.
- **retriever.py (execute_search)**
 - **Function:** Pulls candidate chunks using your PostgreSQL hybrid search (pgvector + tsvector) and applies Voyage AI cross-encoder reranking.
 - **Logic:** Executes the current pending sub-query, filters out results below the reranker threshold (e.g., < 0.50), and appends new chunk IDs to the state to avoid redundant reads.
- **evaluator.py (extract_and_audit)**
 - **Function:** Parses raw retrieved text into structured EvidenceFact objects using Gemini 1.5 Pro with structured outputs.
 - **Logic:** Identifies claims, values, dates, and locations. Checks whether the retrieved passages actually answer the targeted sub-query.
- **arbitrator.py (arbitrate_conflicts)**
 - **Function:** Handles source contradictions (e.g., text stating *"popular accounts wrongly claim otherwise"*).

- **Epistemic Hierarchy Rules:**

1. Canonical codices (Codex Vaeloria) carry weight 1.0.
2. Field annotations and custodial records carry weight 0.8.
3. Popular accounts / folklore carry weight 0.3.

- **Logic:** When two facts conflict (e.g., forging year or threat rating), the higher-tier source wins. The conflict is recorded in conflicts alongside the explanation.

- **synthesizer.py (build_dossier)**

- **Function:** Compiles the final response into a structured Dossier.
- **Logic:** Formulates the final narrative answer, appends an audit trail of how contradictions were arbitrated, and includes citations with links to extracted asset images (Track 1A synergy).

3. Agent Tools (agents/shared/tools.py)

Expose discrete tools to the nodes:

- **hybrid_archive_search(query: str, limit: int = 5):**
 - Executes PostgreSQL RRF search + Voyage reranker.
 - Returns chunk text, source document metadata, and linked visual asset paths.
- **lookup_asset_by_plate(plate_name: str):**
 - Fetches the cropped .png and VLM transcription from media_assets when a text chunk contains a *"consult the plate"* instruction.
- **inspect_visual_detail(asset_path: str, prompt: str):**
 - Sends a live multimodal inspection call to Vertex AI Gemini 1.5 Pro to verify fine visual details (e.g., reading exact ticks on a gauge).

4. Graph Construction (agents/graphs/investigator_1c.py)

Wire the state machine using LangGraph:

```
from langgraph.graph import StateGraph, END
from agents.state.models import InvestigatorState
from agents.nodes.planner import plan_investigation
from agents.nodes.retriever import execute_search
from agents.nodes.evaluator import extract_and_audit
from agents.nodes.arbitrator import arbitrate_conflicts
from agents.nodes.synthesizer import build_dossier

def should_continue(state: InvestigatorState) -> str:
    if state["is_complete"] or state["loop_count"] >= state["max_loops"]:
        return "synthesize"
    return "retrieve"

workflow = StateGraph(InvestigatorState)

workflow.add_node("plan", plan_investigation)
workflow.add_node("retrieve", execute_search)
workflow.add_node("evaluate", extract_and_audit)
workflow.add_node("arbitrate", arbitrate_conflicts)
workflow.add_node("synthesize", build_dossier)

workflow.set_entry_point("plan")
workflow.add_edge("plan", "retrieve")
workflow.add_edge("retrieve", "evaluate")
workflow.add_edge("evaluate", "arbitrate")

workflow.add_conditional_edges(
    "arbitrate",
    should_continue,
    {
        "retrieve": "retrieve",
        "synthesize": "synthesize"
    }
)
```

```
workflow.add_edge("synthesize", END)

investigator_graph = workflow.compile()
```

Track 1A & 1C Integration

By combining 1A and 1C:

1. **1A Ingestion:** Plates, gauges, and diagrams are cropped, transcribed by Gemini Flash, and stored in `media_assets` with local file paths.
 2. **1C Execution:** When a query hits a contradiction or requires multi-step deduction, the 1C investigator loops through the database. If it encounters a chunk referencing a visual plate, it invokes `lookup_asset_by_plate` to inspect the visual data directly.
 3. **Frontend Output:** The final dossier delivers a synthesized answer, an audit log showing why certain claims were rejected, and the exact cropped plate images rendered inline.
-

User prompt: in here do i have memery? summerization? chat history?

Response: In the current schema we planned, **no**—the graph was set up strictly as an **ephemeral execution** (one query in, one synthesized dossier out, then the state is cleared).

If you want the assistant to behave like an interactive conversational agent that remembers earlier turns, answers follow-up questions (e.g., *"What were the threat ratings of those two creatures again?"*), and prevents token blowup via rolling summarization, you need to add **Persistence (Checkpoint)**, **Chat History**, and a **Summarization Node**.

Because PostgreSQL is already in your stack, you don't need any external services—Postgres handles the checkpointer and session persistence natively.

The Three Layers to Add

- **Chat History (Thread-Scoped Memory):** Uses LangGraph's native `add_messages` reducer to append human and AI messages under a distinct session `thread_id`.
- **Persistence (AsyncPostgresSaver):** Saves the exact graph state into PostgreSQL tables after every step, so user conversations persist across browser refreshes.
- **Summarization Loop:** When the message list grows past a threshold (e.g., 6–8 messages), an internal node uses Gemini Flash to compress older turns into a concise summary string while pruning raw tokens.

1. Updated State Schema (`backend/agents/state/models.py`)

Add conversational message channels, the checkpointer metadata, and a persistent summary string to the LangGraph state:

```
from typing import Annotated, TypedDict
from langchain_core.messages import BaseMessage
from langgraph.graph.message import add_messages

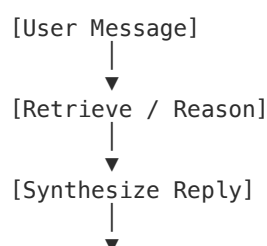
class ConversationalState(TypedDict):
    # Appends new messages and handles RemoveMessage operations automatically
    messages: Annotated[list[BaseMessage], add_messages]

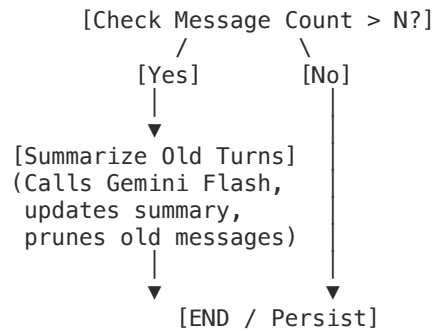
    # Running compressed history of older turns
    conversation_summary: str

    # Core RAG / Investigator fields
    retrieved_chunk_ids: list[str]
    established_facts: list[dict]
    is_complete: bool
```

2. The Summarization & Chat Graph Flow

Wire a conditional edge after the synthesis node that inspects the length of messages:





The Summarize Node Implementation (backend/agents/nodes/summarizer.py):

```

from langchain_core.messages import RemoveMessage, SystemMessage
from backend.agents.llm import get_fast_llm

async def summarize_conversation(state: ConversationalState) -> dict:
    llm = get_fast_llm() # Vertex AI Gemini Flash
    messages = state["messages"]
    existing_summary = state.get("conversation_summary", "")

    # Keep the last 2 turns as immediate active context, compress older messages
    to_summarize = messages[:-2]

    prompt = (
        f"Extend the existing summary of this conversation with the new messages below.\n"
        f"Existing summary: {existing_summary}\n"
        f"New interactions: {to_summarize}\n"
        f"Keep all specific names, threat values, locations, and attunement numbers."
    )

    new_summary = (await llm.ainvoke([SystemMessage(content=prompt)]).content

    # Prune old messages from state using LangGraph's RemoveMessage
    prune_operations = [RemoveMessage(id=m.id) for m in to_summarize]

    return {
        "conversation_summary": new_summary,
        "messages": prune_operations
    }

```

3. Compiling the Graph with PostgreSQL Persistence (backend/agents/graphs/)

Initialize AsyncPostgresSaver pointing to your existing database connection pool:

```

from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver
from backend.core.database import DATABASE_URL

async def get_compiled_agent():
    # Automatically creates required checkpoint tables in Postgres
    async with AsyncPostgresSaver.from_conn_string(DATABASE_URL) as checkpointer:
        await checkpointer.setup()

        compiled_graph = workflow.compile(
            checkpointer=checkpointer
        )
        return compiled_graph

```

4. Running a Conversational Turn via FastAPI (backend/agents/router.py)

Each user session sends a thread_id. LangGraph automatically reloads the conversation history and summary from PostgreSQL:

```

@router.post("/chat")
async def chat_endpoint(thread_id: str, query: str):
    graph = await get_compiled_agent()

    config = {"configurable": {"thread_id": thread_id}}
    input_payload = {"messages": [("user", query)]}

    # Executes graph, auto-loads prior memory, updates summary, and saves to Postgres
    output = await graph.ainvoke(input_payload, config=config)

```

```

return {
    "reply": output["messages"][-1].content,
    "active_summary": output.get("conversation_summary")
}

```

This ensures multi-turn conversational persistence and strict token bounds, all contained within your existing PostgreSQL instance without extra services.

For a walkthrough of implementing summarization nodes and checkpointers in LangGraph, check out the [LangGraph Intro: Optimize Chatbot Memory with Summarization](#) tutorial. This video demonstrates how to set up memory management and trim conversational state so conversations remain efficient over time.

http://googleusercontent.com/youtube_content/1

User prompt: does it make things complex? and explored context? can we limit what we pass? like previous gatherings all we dont need right?

Response: You do not need to pass previous retrieval context back and forth, and doing so is actually an anti-pattern. Accumulating every retrieved chunk into your LLM prompt across conversational turns causes **context bloat**, **prompt dilution**, and **ballooning token costs**.

You do not need an elaborate multi-node summarization system to keep things clean. You can enforce a strict **Context Hygiene** pattern.

The Separation: Ephemeral vs. Persistent State

Separate the state into two distinct categories:

1. **Conversation History (Persistent):** Only stores human messages and final AI answers (e.g., ["User: What is the threat of Weeping Lurker?", "AI: It is 3 per the Vanguard scale."]).
2. **Retrieval Scratchpad (Ephemeral / Overwrite):** Holds raw text chunks and figure paths. This gets completely wiped or overwritten on every new turn.

Turn 1: "What is Weeping Lurker's threat?"

- Retrieve: 5 chunks about Weeping Lurker (5,000 tokens)
- LLM answers: "Threat rating 3"
- WIPE the 5 chunks from memory. Keep ONLY the 1-sentence answer.

Turn 2: "Where does it lair?"

- LLM sees history: "User asked threat, AI said 3."
- Retrieve: 3 NEW chunks about Embermarch (3,000 tokens)
- Old chunks from Turn 1 are NOT passed to the model.

How to Implement This Without Reducers

By default, in LangGraph, if you don't use `operator.add` or an append reducer on a state field, returning that key from a node **completely overwrites** the old value.

In `backend/agents/state/models.py`, leave your retrieval context unreduced:

```

from typing import Annotated, TypedDict
from langchain_core.messages import BaseMessage
from langgraph.graph.message import add_messages

class CleanInvestigatorState(TypedDict):
    # Only messages use an append reducer (tracks the chat thread)
    messages: Annotated[list[BaseMessage], add_messages]

    # EPHEMERAL: Overwritten on every new turn/iteration (No operator.add!)
    active_query: str
    retrieved_chunks: list[dict] # Wiped clean each turn
    active_figures: list[str]    # Wiped clean each turn

    # PERSISTENT: Lightweight extracted facts only
    verified_facts: list[str]    # e.g., ["Weeping Lurker: Threat 3"]

```

How to Trim History Without Complex Summarizer Nodes

Instead of building an extra LLM summarizer node that burns calls on every turn, use LangChain's built-in message trimmer. It keeps the prompt under a strict sliding window limit right before the LLM call:

```

from langchain_core.messages import trim_messages
from backend.agents.llm import get_gemini_pro

```

```

async def synthesize_node(state: CleanInvestigatorState) -> dict:
    llm = get_gemini_pro()

    # Keep only the last 3-4 chat messages to maintain context without bloat
    trimmed_chat = trim_messages(
        state["messages"],
        max_tokens=1000,
        strategy="last",
        token_counter=llm,
        start_on="human"
    )

    # Pass ONLY:
    # 1. Trimmed chat history (a few hundred tokens)
    # 2. Fresh chunks from the CURRENT retrieval turn only
    prompt = [
        {"role": "system", "content": "Answer using ONLY the current evidence."},
        *trimmed_chat,
        {"role": "user", "content": f"Current Evidence: {state['retrieved_chunks']}"}
    ]

    response = await llm.ainvoke(prompt)

    return {
        "messages": [response],
        # Explicitly clear out raw chunk data so it doesn't linger in DB checkpoints
        "retrieved_chunks": [],
        "active_figures": []
    }

```

What This Solves

- **No Exploding Context:** Each turn only consumes the tokens of the latest search plus a tiny 2-to-3 message window.
- **No Stale Hallucinations:** The model won't confuse chunks retrieved for *Greyfell Citadel* in turn 1 with chunks retrieved for *Marrowwell Abbey* in turn 2.
- **Minimal Complexity:** You do not need a summarization agent, custom cron jobs, or complex message-slicing logic. The state naturally resets its retrieval buffers after every output.

User prompt: *but we need some kind of idea right? how we preserve that without exploding context*

Response: To preserve conversational continuity without causing token bloat, the industry-standard architecture relies on **Contextual Query Rewriting (Condense Question)** paired with a **Lightweight Facts Registry**.

Instead of accumulating thousands of tokens of raw text chunks from turn to turn, you only carry forward two compact elements:

- **A sliding window of chat messages** (the raw user prompts and concise final AI answers, typically under 500 tokens).
- **The query rewriter** that resolves pronouns and references *before* hitting the database.

The Problem It Solves

Turn 1:

User: "What is the attunement cost of The Thrice-Bound Edge?"

Retriever fetches: 4 pages of Codex Vaeloria II (3,500 tokens).

AI outputs: "The attunement cost is 94 vitae-grains, per the Vanguard plate."

Turn 2:

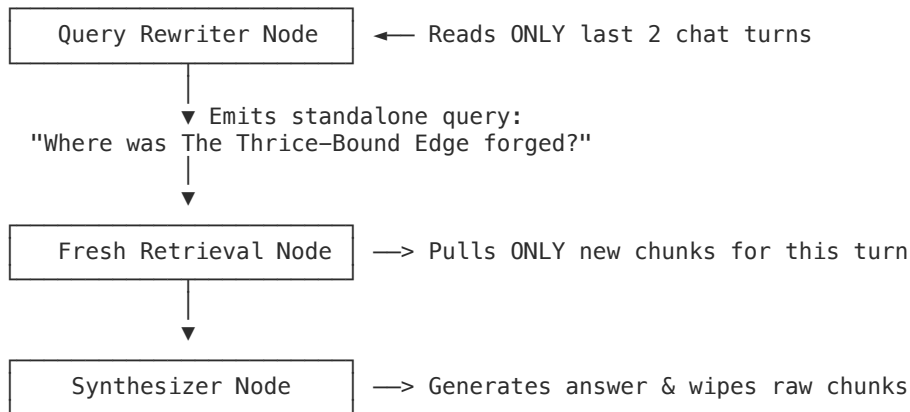
User: "Where was it forged?"

- **If you don't carry context:** The database searches for "Where was it forged?" and returns zero relevant results because it doesn't know what *"it"* refers to.
- **If you carry everything:** You send 7,000+ tokens of prior retrieved text chunks, risking context confusion and wasted API cost.
- **The Solution (Query Rewriting):** A fast LLM converts "Where was it forged?" into "Where was The Thrice-Bound Edge forged?" *before* executing the search.

The Minimal Flow

[User Input: "Where was it forged?"]





1. The State Schema (backend/agents/state/models.py)

Keep the shared state lean. Do not accumulate chunks:

```

from typing import Annotated, TypedDict
from langchain_core.messages import BaseMessage
from langgraph.graph.message import add_messages

class ConversationalInvestigatorState(TypedDict):
    # Appends new conversational messages only (User + AI text)
    messages: Annotated[list[BaseMessage], add_messages]

    # Standalone search query generated by the rewriter
    search_query: str

    # Ephemeral: fresh chunks for the CURRENT turn only (overwritten each turn)
    active_chunks: list[dict]
    active_figures: list[str]
  
```

2. The Query Rewriter Node (backend/agents/nodes/rewriter.py)

A 50-token call to Gemini Flash resolves ambiguous references:

```

from langchain_core.messages import SystemMessage, HumanMessage
from backend.agents.llm import get_fast_llm
from backend.agents.state.models import ConversationalInvestigatorState

async def rewrite_query(state: ConversationalInvestigatorState) -> dict:
    # If this is the first turn, no rewriting is required
    if len(state["messages"]) <= 1:
        return {"search_query": state["messages"][-1].content}

    llm = get_fast_llm() # Gemini 1.5/2.5 Flash

    # Provide only the last 2-3 dialogue turns for context
    recent_history = state["messages"][-3:]

    prompt = (
        "Given the conversation history and the latest user inquiry, "
        "reformulate the latest question into a standalone, fully-qualified search query. "
        "Resolve all pronouns ('it', 'they', 'that relic', 'the creature') using the history. "
        "Do NOT answer the question; return ONLY the rewritten query text."
    )

    response = await llm.ainvoke([
        SystemMessage(content=prompt),
        *recent_history
    ])

    return {"search_query": response.content.strip()}
  
```

3. The Retrieval and Synthesis Nodes

1. **retriever.py**: Executes your PostgreSQL hybrid search and Voyage reranker using `state["search_query"]`. It returns `active_chunks` for this turn only.
2. **synthesizer.py**: Passes `state["search_query"]` and `state["active_chunks"]` to Gemini 1.5 Pro to generate the final response.

```

async def synthesize_response(state: ConversationalInvestigatorState) -> dict:
    llm = get_gemini_pro()

    prompt = f"""
    Context from Archive:
    {state['active_chunks']}

    User Query: {state['search_query']}

    Provide a concise, grounded response with exact citations.
    """

    response = await llm.ainvoke(prompt)

    # The message list gets the AI's final answer appended.
    # active_chunks is reset to an empty list so it does not persist in the checkpoint.
    return {
        "messages": [response],
        "active_chunks": [],
        "active_figures": []
    }

```

Why This Architecture Stays Fast and Scalable

- **Bounded Prompt Size:** No matter how long the user chats (1 turn or 40 turns), each retrieval call only processes the latest user query, and the synthesizer only receives 3–5 fresh chunks.
- **Accurate Vector Embeddings:** Passing "Where was it forged?" into Voyage AI yields weak semantic matches. Passing the rewritten query "Where was The Thrice-Bound Edge forged?" returns precise vectors and matches BM25 keywords exactly.
- **Zero Cross-Contamination:** Chunks retrieved for a monster in Turn 1 will not pollute or contradict chunks retrieved for a castle in Turn 2.

User prompt: cool lets give plan,

Response: Implementation Roadmap

Phase 1: Environment & Base Infrastructure

- Initialize dependency environment with uv (pyproject.toml, uv.lock) specifying:
 - fastapi, uvicorn, pydantic-settings
 - sqlalchemy[asyncio], asyncpg, pgvector
 - docling, pillow, imagehash, pypdfium2
 - voyageai, google-genai, langchain-google-vertexai, langgraph
- Create docker-compose.yml defining PostgreSQL with the pgvector/pgvector:pg16 image and standard volume mounts.
- Set up core/config.py using pydantic-settings to load GCP Vertex credentials, Voyage API keys, and database connection strings.
- Implement database session handling in core/database.py and run the bootstrap DDL to activate vector extension and declare schemas:
 - media_assets (id, sha256, phash, file_path, vlm_transcription, metadata_payload)
 - document_chunks (id, doc_id, content, embedding, tsv, asset_id, metadata_payload)

Phase 2: Polymorphic Ingestion Worker (workers/run_ingest.py)

- **Asset Deduplication (ingestion/image_dedup.py):**
 - Implement double-check hash lookup: SHA-256 for exact match, perceptual hash (imagehash.dhash) with Hamming distance ≤ 2 for compressed/resized duplicates.
- **Layout Parsing & VLM Transcription (ingestion/parser.py, ingestion/image_vlm.py):**
 - **Markdown files:** Parse inline syntax `![label](path)`, register local image files, and attach to entity headers.

- **Codex PDFs:** Process via IBM Docling, isolate bounding boxes for tables and Matplotlib plates, crop images via Pillow to data/extracted_assets/, and call Vertex AI Gemini 1.5 Flash to transcribe gauge metrics and bar charts into explicit key-value figures.
- **Scanned Ephemera (*.scan.pdf):** Convert full pages to 300 DPI images via pypdfium2, run full-page OCR transcription with Gemini Flash.
- **Chunking & Hybrid Indexing (ingestion/chunker.py, ingestion/embedder.py):**
 - Merge text passages with corresponding VLM transcribed figures and metadata paths.
 - Generate 1024-dimensional embeddings using Voyage AI (voyage-multimodal-3.5) with exponential backoff (tenacity).
 - Populate PostgreSQL tables (document_chunks with embedding and generated tsvector).

Phase 3: Hybrid Retrieval & Reranking Core (retrieval/)

- **Hybrid SQL Engine (retrieval/vector_store.py):**
 - Write a single SQL query implementing Reciprocal Rank Fusion (RRF) combining vector cosine distance ($\leq$) on HNSW indexes and text search rank (ts_rank_cd) on tsvector.
- **Cross-Encoder Reranker (retrieval/reranker.py):**
 - Accept top 30-50 retrieved candidates and score via Voyage AI rerank-2.5.
 - Enforce confidence score cutoff threshold (≥ 0.50) to drop noise before passing context upstream.

Phase 4: LangGraph Agent & Conversational Engine (agents/)

- **State Modeling (agents/state/models.py):**
 - Configure ConversationalInvestigatorState containing persistent message history, standalone rewritten search_query, ephemeral active_chunks, and active_figures.
- **Node Implementation (agents/nodes/):**
 - `rewriter.py`: Call Gemini Flash on the last 2-3 dialogue turns to resolve pronouns and produce standalone retrieval queries.
 - `retriever.py`: Call retrieval/vector_store.py and populate current turn's active_chunks and figure paths.
 - `arbitrator.py` (Track 1C): Compare claims across retrieved chunks, applying source credibility weights (e.g., official codices over popular accounts) to resolve conflicting lore.
 - `synthesizer.py`: Pass current question, verified chunks, and local image paths to Vertex AI Gemini 1.5 Pro with structured Pydantic output (answer_text, referenced_figures). Wipe active_chunks on completion.
- **Workflow Compilation (agents/graphs/):**
 - Assemble graph edges: `rewriter` → `retriever` → `arbitrator` → `synthesizer` → END.
 - Bind state persistence using AsyncPostgresSaver backed by PostgreSQL.

Phase 5: API Integration & Frontend Delivery

- Configure FastAPI app (`main.py`):
 - Mount /assets endpoint to data/extracted_assets/ using StaticFiles.
 - Expose POST /api/chat accepting thread_id and query string.
 - Return strict JSON payloads linking answer text directly to local figure URLs (/assets/...).
 - Verify with integration tests using codex_vaeloria_ii test cases:
 - Validate Threat Rating extracted as 4 for Marsh Revenant.
 - Validate Attunement Cost extracted as 94 for The Thrice-Bound Edge.
 - Ensure subsequent follow-up queries ("Where was it forged?") correctly rewrite the prompt without expanding token history.
-